

数据清洗及特征处理

2026 年 5 月 20 日

1 数据清洗及特征处理

开始之前，导入 NumPy、Pandas 包和数据

```
[1]: # 加载所需的库
import numpy as np
import pandas as pd

[2]: # 加载数据 train.csv
df = pd.read_csv('/home/jovyan/work/datasets/6645a647c4db0a94029f0bf5-momodel/
↳dataset/train.csv')
df.head(3)
```

```
[2]: PassengerId  Survived  Pclass  \
0             1         0         3
1             2         1         1
2             3         1         3

                                     Name    Sex  Age  SibSp  \
0                        Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                        Heikkinen, Miss. Laina  female  26.0      0

    Parch    Ticket   Fare Cabin Embarked
0       0   A/5 21171   7.2500   NaN        S
1       0   PC 17599  71.2833   C85        C
2       0 STON/O2. 3101282   7.9250   NaN        S
```

数据清洗简述:

我们拿到的数据通常是不干净的，所谓的不干净，就是数据中有缺失值，有一些异常点等，需要经过一定的处理才能继续做后面的分析或建模，所以拿到数据的第一步是进行数据清洗。

本章我们将学习缺失值、重复值、字符串和数据转换等操作，将数据清洗成可以分析或建模的样子。

1.1 1. 缺失值观察与处理

我们拿到的数据经常会有很多缺失值，比如我们可以看到 Cabin 列存在 NaN，那其他列还有没有缺失值，这些缺失值要怎么处理呢？

任务一：缺失值观察

(1) 请查看每个特征缺失值个数

(2) 请查看 Age, Cabin, Embarked 列的数据

以上方式都有多种方式，所以建议大家学习的时候多多益善

```
[3]: # 方法一
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   PassengerId     891 non-null   int64
 1   Survived        891 non-null   int64
 2   Pclass          891 non-null   int64
 3   Name            891 non-null   object
 4   Sex             891 non-null   object
 5   Age             714 non-null   float64
 6   SibSp           891 non-null   int64
 7   Parch           891 non-null   int64
 8   Ticket          891 non-null   object
 9   Fare            891 non-null   float64
10   Cabin           204 non-null   object
11   Embarked        889 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

```
[4]: # 方法二
df.isnull().sum()
```

```
[4]: PassengerId      0
      Survived        0
      Pclass         0
      Name           0
      Sex            0
      Age           177
      SibSp          0
      Parch          0
      Ticket         0
      Fare           0
      Cabin         687
      Embarked       2
      dtype: int64
```

```
[5]: df[['Age', 'Cabin', 'Embarked']].head(3)
```

```
[5]:   Age Cabin Embarked
0  22.0  NaN         S
1  38.0   C85         C
2  26.0  NaN         S
```

任务二：对缺失值进行处理

(1) 处理缺失值一般有几思路

(2) 请尝试对 Age 列的数据的缺失值进行处理

(3) 请尝试使用不同的方法直接对整张表的缺失值进行处理

以下是举例：

```
[6]: df[df['Age']==None]=0
df.head(3)
```

```
[6]:   PassengerId  Survived  Pclass  \
0             1         0        3
```

```
1      2      1      1
2      3      1      3
```

```

                                Name      Sex  Age  SibSp  \
0                        Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th... female  38.0      1
2                        Heikkinen, Miss. Laina  female  26.0      0

```

```

      Parch      Ticket    Fare Cabin Embarked
0         0      A/5 21171    7.2500   NaN        S
1         0      PC 17599   71.2833   C85        C
2         0  STON/O2. 3101282    7.9250   NaN        S

```

```
[7]: df[df['Age'].isnull()] = 0 # 还好
df.head(3)
```

```
[7]: PassengerId  Survived  Pclass  \
0             1         0        3
1             2         1        1
2             3         1        3
```

```

                                Name      Sex  Age  SibSp  \
0                        Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th... female  38.0      1
2                        Heikkinen, Miss. Laina  female  26.0      0

```

```

      Parch      Ticket    Fare Cabin Embarked
0         0      A/5 21171    7.2500   NaN        S
1         0      PC 17599   71.2833   C85        C
2         0  STON/O2. 3101282    7.9250   NaN        S

```

```
[8]: df[df['Age'] == np.nan] = 0
df.head()
```

```
[8]: PassengerId  Survived  Pclass  \
0             1         0        3
1             2         1        1
```

2	3	1	3
3	4	1	1
4	5	0	3

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

【思考】检索空缺值用 `np.nan`, `None` 以及 `.isnull()` 哪个更好，这是为什么？如果其中某个方式无法找到缺失值，原因又是为什么？

【回答】数值列读取数据后，空缺值的数据类型为 `float64` 所以用 `None` 一般索引不到，比较的时候最好用 `np.nan`

```
[9]: df.dropna().head(3)
```

```
[9]: PassengerId  Survived  Pclass  \
1             2         1         1
3             4         1         1
5             0         0         0
```

	Name	Sex	Age	SibSp	\
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
5		0	0	0.0	0

	Parch	Ticket	Fare	Cabin	Embarked
1	0	PC 17599	71.2833	C85	C
3	0	113803	53.1000	C123	S
5	0	0	0.0000	0	0

```
[10]: df.fillna(0).head(3)
```

```
[10]: PassengerId  Survived  Pclass  \
0             1         0         3
1             2         1         1
2             3         1         3
```

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	0	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	0	S

【思考】dropna 和 fillna 有哪些参数，分别如何使用呢？

【参考】<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.dropna.html>

【参考】<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.fillna.html>

1.2 2. 重复值观察与处理

数据中会不会存在重复值呢，如果存在要怎样处理呢

任务一：请查看数据中的重复值

```
[11]: df[df.duplicated()]
```

```
[11]: PassengerId  Survived  Pclass  Name  Sex  Age  SibSp  Parch  Ticket  Fare  \
17             0         0         0    0    0  0.0        0        0        0  0.0
19             0         0         0    0    0  0.0        0        0        0  0.0
```

26	0	0	0	0	0	0.0	0	0	0	0.0
28	0	0	0	0	0	0.0	0	0	0	0.0
29	0	0	0	0	0	0.0	0	0	0	0.0
..	...	...	...	...	...	...	...	...	...	...
859	0	0	0	0	0	0.0	0	0	0	0.0
863	0	0	0	0	0	0.0	0	0	0	0.0
868	0	0	0	0	0	0.0	0	0	0	0.0
878	0	0	0	0	0	0.0	0	0	0	0.0
888	0	0	0	0	0	0.0	0	0	0	0.0

	Cabin	Embarked
17	0	0
19	0	0
26	0	0
28	0	0
29	0	0
..	...	...
859	0	0
863	0	0
868	0	0
878	0	0
888	0	0

[176 rows x 12 columns]

任务二：对重复值进行处理

(1) 重复值有哪些处理方式呢？

(2) 处理我们数据的重复值

以下是对整个行有重复值的清理的方法举例：

```
[12]: df = df.drop_duplicates()
      df.head()
```

```
[12]: PassengerId  Survived  Pclass  \
0             1         0         3
1             2         1         1
```

2	3	1	3
3	4	1	1
4	5	0	3

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

任务三：将前面清洗的数据保存为 csv 格式

```
[13]: df.to_csv('/home/jovyan/work/results/test_clear.csv')
```

1.3 3. 特征观察与处理

我们对特征进行一下观察，可以把特征大概分为两大类：

数值型特征：Survived，Pclass，Age，SibSp，Parch，Fare，

其中 Survived，Pclass 为离散型数值特征，Age，SibSp，Parch，Fare 为连续型数值特征

文本型特征：Name，Sex，Cabin，Embarked，Ticket，

其中 Sex，Cabin，Embarked，Ticket 为类别型文本特征。

数值型特征一般可以直接用于模型的训练，但有时候为了模型的稳定性及鲁棒性会对连续变量进行离散化。

文本型特征往往需要转换成数值型特征才能用于建模分析。

任务一：对年龄进行分箱（离散化）处理

(1) 分箱操作是什么？

(2) 将连续变量 Age 平均分箱成 5 个年龄段，并分别用类别变量 12345 表示

(3) 将连续变量 Age 划分为 (0,5] (5,15] (15,30] (30,50] (50,80] 五个年龄段，并分别用类别变量 12345 表示

(4) 将连续变量 Age 按 10% 30% 50% 70% 90% 五个年龄段，并用分类变量 12345 表示

(5) 将上面的获得的数据分别进行保存，保存为 csv 格式

```
[14]: # 将连续变量 Age 平均分箱成 5 个年龄段，并分别用类别变量 12345 表示
df['AgeBand'] = pd.cut(df['Age'], 5, labels = [1,2,3,4,5])
df.head()
```

```
[14]:
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked	AgeBand
0	0	A/5 21171	7.2500	NaN	S	2
1	0	PC 17599	71.2833	C85	C	3
2	0	STON/O2. 3101282	7.9250	NaN	S	2
3	0	113803	53.1000	C123	S	3
4	0	373450	8.0500	NaN	S	3

```
[15]: df.to_csv('/home/jovyan/work/results/test_ave.csv')
```

```
[16]: # 将连续变量 Age 划分为 (0,5] (5,15] (15,30] (30,50] (50,80] 五个年龄段, 并分别用
      类别变量 12345 表示
      df['AgeBand'] = pd.cut(df['Age'],[0,5,15,30,50,80],labels = [1,2,3,4,5])
      df.head(3)
```

```
[16]: PassengerId  Survived  Pclass  \
0             1           0         3
1             2           1         1
2             3           1         3

      Name      Sex  Age  SibSp  \
0      Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2      Heikkinen, Miss. Laina    female  26.0      0

      Parch      Ticket    Fare Cabin Embarked AgeBand
0         0      A/5 21171    7.2500   NaN        S        3
1         0      PC 17599   71.2833   C85        C        4
2         0  STON/O2. 3101282    7.9250   NaN        S        3
```

```
[17]: df.to_csv('/home/jovyan/work/results/test_cut.csv')
```

```
[18]: # 将连续变量 Age 按 10% 30% 50 70% 90% 五个年龄段, 并用分类变量 12345 表示
      df['AgeBand'] = pd.qcut(df['Age'],[0,0.1,0.3,0.5,0.7,0.9],labels = [1,2,3,4,5])
      df.head()
```

```
[18]: PassengerId  Survived  Pclass  \
0             1           0         3
1             2           1         1
2             3           1         3
3             4           1         1
4             5           0         3

      Name      Sex  Age  SibSp  \
0      Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2      Heikkinen, Miss. Laina    female  26.0      0
```

```

3      Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4      Allen, Mr. William Henry      male  35.0      0

```

```

      Parch      Ticket    Fare Cabin Embarked AgeBand
0         0    A/5 21171    7.2500   NaN        S        2
1         0      PC 17599   71.2833   C85        C        5
2         0  STON/O2. 3101282    7.9250   NaN        S        3
3         0      113803   53.1000  C123        S        4
4         0      373450    8.0500   NaN        S        4

```

```
[19]: df.to_csv('/home/jovyan/work/results/test_pr.csv')
```

【参考】 <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.cut.html>

【参考】 <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.qcut.html>

任务二：对文本变量进行转换

(1) 查看文本变量名及种类

(2) 将文本变量 Sex, Cabin, Embarked 用数值变量 12345 表示

(3) 将文本变量 Sex, Cabin, Embarked 用 one-hot 编码表示

```
[20]: # 查看类别文本变量名及种类
```

```

# 方法一: value_counts
df['Sex'].value_counts()

```

```

[20]: male      453
      female    261
      0         1
      Name: Sex, dtype: int64

```

```
[21]: df['Cabin'].value_counts()
```

```

[21]: G6      4
      C23 C25 C27  4
      B96 B98    4
      F33      3
      C22 C26    3
      ..

```

```

C124      1
C32       1
E34       1
C7        1
C148      1
Name: Cabin, Length: 135, dtype: int64

```

```
[22]: df['Embarked'].value_counts()
```

```

[22]: S    554
      C    130
      Q     28
      O      1
      Name: Embarked, dtype: int64

```

```

[23]: # 方法二: unique
      df['Sex'].unique()

```

```
[23]: array(['male', 'female', 0], dtype=object)
```

```
[24]: df['Sex'].nunique()
```

```
[24]: 3
```

```

[25]: # 将类别文本转换为 12345

      # 方法一: replace
      df['Sex_num'] = df['Sex'].replace(['male', 'female'], [1, 2])
      df.head()

```

```

[25]:   PassengerId  Survived  Pclass  \
0             1         0         3
1             2         1         1
2             3         1         3
3             4         1         1
4             5         0         3

```

```

      Name    Sex  Age  SibSp  \

```

0		Braund, Mr. Owen Harris	male	22.0	1
1		Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1
2		Heikkinen, Miss. Laina	female	26.0	0
3		Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1
4		Allen, Mr. William Henry	male	35.0	0

	Parch	Ticket	Fare	Cabin	Embarked	AgeBand	Sex_num
0	0	A/5 21171	7.2500	NaN	S	2	1
1	0	PC 17599	71.2833	C85	C	5	2
2	0	STON/O2. 3101282	7.9250	NaN	S	3	2
3	0	113803	53.1000	C123	S	4	2
4	0	373450	8.0500	NaN	S	4	1

```
[26]: # 方法二: map
df['Sex_num'] = df['Sex'].map({'male': 1, 'female': 2})
df.head()
```

```
[26]: PassengerId  Survived  Pclass  \
0             1         0         3
1             2         1         1
2             3         1         3
3             4         1         1
4             5         0         3
```

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked	AgeBand	Sex_num
0	0	A/5 21171	7.2500	NaN	S	2	1.0
1	0	PC 17599	71.2833	C85	C	5	2.0
2	0	STON/O2. 3101282	7.9250	NaN	S	3	2.0
3	0	113803	53.1000	C123	S	4	2.0
4	0	373450	8.0500	NaN	S	4	1.0

```
[27]: # 方法三: 使用 sklearn.preprocessing 的 LabelEncoder
from sklearn.preprocessing import LabelEncoder
for feat in ['Cabin', 'Ticket']:
    lbl = LabelEncoder()
    label_dict = dict(zip(df[feat].unique(), range(df[feat].nunique())))
    df[feat + "_labelEncode"] = df[feat].map(label_dict)
    df[feat + "_labelEncode"] = lbl.fit_transform(df[feat].astype(str))

df.head()
```

```
[27]: PassengerId  Survived  Pclass  \
0             1         0         3
1             2         1         1
2             3         1         3
3             4         1         1
4             5         0         3
```

```

                                Name      Sex  Age  SibSp  \
0                        Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                        Heikkinen, Miss. Laina  female  26.0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                        Allen, Mr. William Henry    male  35.0      0
```

```

    Parch      Ticket    Fare Cabin Embarked AgeBand  Sex_num  \
0      0    A/5 21171    7.2500   NaN      S        2      1.0
1      0    PC 17599   71.2833   C85      C        5      2.0
2      0  STON/O2. 3101282    7.9250   NaN      S        3      2.0
3      0    113803   53.1000  C123      S        4      2.0
4      0    373450    8.0500   NaN      S        4      1.0
```

```

    Cabin_labelEncode  Ticket_labelEncode
0                  135                  409
1                   74                  472
2                  135                  533
3                   50                   41
```

4

135

374

[28]: # 将类别文本转换为 *one-hot* 编码

```
# 方法一: OneHotEncoder
for feat in ["Age", "Embarked"]:
    # x = pd.get_dummies(df["Age"] // 6)
    # x = pd.get_dummies(pd.cut(df['Age'],5))
    x = pd.get_dummies(df[feat], prefix=feat)
    df = pd.concat([df, x], axis=1)
    #df[feat] = pd.get_dummies(df[feat], prefix=feat)

df.head()
```

[28]: PassengerId Survived Pclass \

```
0      1      0      3
1      2      1      1
2      3      1      3
3      4      1      1
4      5      0      3
```

```

                                Name      Sex  Age  SibSp  \
0                                Braund, Mr. Owen Harris  male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                                Heikkinen, Miss. Laina  female  26.0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                                Allen, Mr. William Henry  male  35.0      0
```

```

    Parch      Ticket    Fare  ... Age_66.0 Age_70.0 Age_70.5 Age_71.0  \
0      0      A/5 21171   7.2500  ...      0      0      0      0
1      0      PC 17599  71.2833  ...      0      0      0      0
2      0  STON/O2. 3101282   7.9250  ...      0      0      0      0
3      0      113803  53.1000  ...      0      0      0      0
4      0      373450   8.0500  ...      0      0      0      0
```

```

    Age_74.0 Age_80.0 Embarked_0 Embarked_C Embarked_Q Embarked_S
0          0          0          0          0          0          1
```

1	0	0	0	1	0	0
2	0	0	0	0	0	1
3	0	0	0	0	0	1
4	0	0	0	0	0	1

[5 rows x 109 columns]

任务三（附加）：从纯文本 Name 特征里提取出 Titles 的特征（所谓的 Titles 就是 Mr, Miss, Mrs 等）

```
[29]: df['Title'] = df.Name.str.extract('([A-Za-z]+)\.', expand=False)
df.head()
```

```
[29]: PassengerId  Survived  Pclass  \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3
```

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	...	Age_70.0	Age_70.5	Age_71.0	Age_74.0	\
0	0	A/5 21171	7.2500	...	0	0	0	0	
1	0	PC 17599	71.2833	...	0	0	0	0	
2	0	STON/O2. 3101282	7.9250	...	0	0	0	0	
3	0	113803	53.1000	...	0	0	0	0	
4	0	373450	8.0500	...	0	0	0	0	

	Age_80.0	Embarked_0	Embarked_C	Embarked_Q	Embarked_S	Title
0	0	0	0	0	1	Mr
1	0	0	1	0	0	Mrs
2	0	0	0	0	1	Miss

3	0	0	0	0	1	Mrs
4	0	0	0	0	1	Mr

[5 rows x 110 columns]

```
[30]: # 保存上面的为最终结论
df.to_csv('/home/jovyan/work/results/test_fin.csv')
```